

CIT-2109 · Arquitectura y Organización de Computadores

Algoritmos Aritméticos

Booth, Multiplicación Sin Signo e IEEE 754

Edicson Solar Salinas

edicson.solar@mail.udp.cl

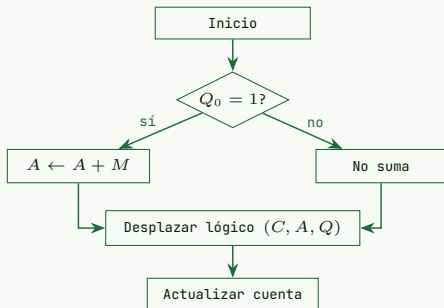
+56 9 92763279

08

19 de mayo de 2026

- Algoritmos de Multiplicación (Sin Signo y Algoritmo de Booth)
- Estructura de Punto Flotante IEEE 754
- Algoritmo de Suma en Punto Flotante
- Ejercicio 1: Análisis Teórico de Pérdida de Precisión
- Ejercicio 2: Trazado de Algoritmo de Booth
- Ejercicio 3: Codificación IEEE 754
- Ejercicio 4: Suma de Códigos en Punto Flotante
- Errores Frecuentes

Multiplicación Sin Signo



Multiplicación Sin Signo: Depende del bit menos significativo del multiplicador (Q_0). Si es 1, $A \leftarrow A + M$. Luego, desplazamiento lógico a la derecha de C, A, Q.

Algoritmo de Booth (Complemento a 2)

Evalúa Q_0 y Q_{-1} simultáneamente.

Q_0Q_{-1}	Operación
10	$A \leftarrow A - M$
01	$A \leftarrow A + M$
00	Sin suma/resta
11	Sin suma/resta

Si $Q_0Q_{-1} = 10 \rightarrow A \leftarrow A - M$.

Si $Q_0Q_{-1} = 01 \rightarrow A \leftarrow A + M$.

Si $Q_0Q_{-1} = 00$ o $11 \rightarrow$ Sin suma/resta. Solo se aplica desplazamiento aritmético a la derecha (A, Q, Q_{-1}), conservando el bit de signo en A.

Formato IEEE 754 Precisión Simple (32 bits):

**Signo (S)**

1 bit (0 = Positivo, 1 = Negativo).

Exponente Sesgado (E)

8 bits (Sesgo = 127).

Parte Significativa / Mantisa (M)

23 bits (Incluye un "1" implícito a la izquierda de la coma).

$$\text{Ecuación del valor: } \pm 1.M \times 2^{E-127}$$

Idea central

No se suman los códigos hexadecimales como enteros. Primero se decodifican los campos IEEE 754 y luego se trabaja con la forma:

$$\pm 1.M \times 2^e$$

Para poder sumar mantisas, ambos operandos deben tener el mismo exponente.

Procedimiento

1. Separar signo, exponente y mantisa.
2. Restar el sesgo: $e = E - 127$.
3. Reinsertar el 1 implícito en la mantisa.
4. Igualar exponentes desplazando a la derecha la mantisa del número con menor exponente.
5. Sumar/restar mantisas según los signos.
6. Normalizar y volver a codificar.

Si $e_A < e_B$, entonces la mantisa de A se desplaza $e_B - e_A$ posiciones a la derecha antes de sumar.

Al realizar una suma/resta en coma flotante de dos números de valor muy próximo, por ejemplo:

$$X = 1.00...00 \times 2^1 \quad Y = 1.11...11 \times 2^0$$

el menor debe desplazarse a la derecha para igualar los exponentes.

Pregunta

¿Por qué la arquitectura de hardware requiere añadir “bits de guarda” temporales durante este desplazamiento?

Ejercicio 2: Trazado de Algoritmo de Booth

Realice la multiplicación de -68 (M) y +41 (Q) utilizando el Algoritmo de Booth con 8 bits.

Complete la inicialización, las operaciones de suma/resta y cada desplazamiento aritmético a la derecha.

Operación	A	Q	Q_{-1}	M	Cuenta
Inicial					
Ciclo 1					
Ciclo 2					
Ciclo 3					
Ciclo 4					
Ciclo 5					
Ciclo 6					
Ciclo 7					
Ciclo 8					

Utilizando el estándar IEEE 754 de precisión simple (32 bits), represente el número decimal -5.

Paso	Resultado
Valor absoluto en binario	
Forma normalizada	
Signo	
Exponente sesgado	
Mantisa almacenada	

Código Final: _____

Ejercicio 4: Suma de Códigos en Punto Flotante

Considerando el algoritmo para sumar números en punto flotante, realice la suma de los siguientes códigos: 0x49CD8000 y 0x4ECB7000.

Operando A (0x49CD8000)

Signo: _____

Exponente: _____

Mantisa con 1 implícito: _____

Operando B (0x4ECB7000)

Signo: _____

Exponente: _____

Mantisa con 1 implícito: _____

Proceso de Suma:

Alinear exponentes: _____

Operando desplazado: _____

Suma de partes significativas:

+ _____

1) En Algoritmo de Booth

Usar desplazamiento lógico cuando debe ser aritmético.

2) En Multiplicación Sin Signo

Sumar sin revisar Q_0 o desplazar aritméticamente.

3) En Punto Flotante

Sumar mantisas sin alinear exponentes, o guardar el 1 implícito dentro de la mantisa.

**DUDAS
CONSULTAS
FUNAS
RECLAMOS
AMENAZAS DE MUERTE**

CUÍDENSE

